

Proposta de Seminario: Projeto de Banco de Dados

Disciplina: Projeto de Banco de Dados

2026

Titulo / Tema

Administracao e Otimizacao de Bancos de Dados Vetoriais para Aplicacoes de Inteligencia Artificial

Equipe

- Jose Inamar de Medeiros Junior

SGBD (Sistema de Gerenciamento de Banco de Dados)

PostgreSQL (utilizando a extensao pgvector)

O PostgreSQL e um robusto Sistema de Gerenciamento de Banco de Dados Objeto-Relacional (ORDBMS) de codigo aberto, amplamente reconhecido por sua confiabilidade, integridade de dados (conformidade ACID) e extensibilidade. Embora seja tradicionalmente utilizado para dados relacionais, sua arquitetura permite a integracao de extensoes poderosas.

Para este seminario, o foco sera no PostgreSQL equipado com a extensao pgvector. Esta extensao transforma o PostgreSQL em um banco de dados vetorial de alto desempenho, capaz de armazenar e consultar embeddings de alta dimensionalidade gerados por modelos de Inteligencia Artificial (como LLMs). O pgvector permite a execucao de buscas por similaridade utilizando metricas Matematicas diretas no banco de dados. Por exemplo, a similaridade de cosseno entre dois vetores A e B e calculada internamente como:

$$similaridade(A, B) = (A \cdot B) / (||A|| * ||B||)$$

Ao utilizar o PostgreSQL, evitamos a necessidade de provisionar uma infraestrutura de banco de dados vetorial separada e dedicada, permitindo que dados relacionais (como informacoes de usuarios ou metadados de projetos) coexistam e sejam consultados simultaneamente com dados vetoriais (embeddings semanticos).

Proposta Pratica

A demonstracao pratica focara na implementacao de um fluxo de Retrieval-Augmented Generation (RAG) para uma aplicacao de Inteligencia Artificial, enfatizando a administracao e a otimizacao das consultas vetoriais no banco de dados. As seguintes etapas serao realizadas:

1. Configuracao e Administracao: Instalacao de uma instancia do PostgreSQL e habilitacao da extensao pgvector. Criacao das tabelas relacionais contendo uma coluna do tipo vetor (vector(n)), onde n representa as dimensoes do embedding de um modelo de IA de codigo aberto.

2. Integracao no Backend: Desenvolvimento de um servico backend (utilizando Go ou Python) que processa documentos de texto, interage com uma API de IA para gerar os embeddings e insere esses vetores no banco de dados.

3. Busca por Similaridade: Implementacao de uma funcionalidade de busca onde uma consulta de texto (prompt do usuario) e convertida em um vetor, e o banco de dados executa uma busca de vizinhos mais proximos (K-Nearest Neighbors - KNN) utilizando operadores do pgvector (como $\leq$ para distancia de cosseno) para retornar os contextos mais relevantes.

4. Otimizacao de Desempenho: A demonstracao pratica culminara na otimizacao do banco de dados. Serao gerados milhares de registros para simular um ambiente de producao. Em seguida, compararemos o tempo de execucao e o custo computacional de buscas exatas (sem indice) versus buscas aproximadas (Approximate Nearest Neighbor - ANN) atraves da criacao e configuracao de indices HNSW (Hierarchical Navigable Small World) no PostgreSQL.

Referencias

- PostgreSQL Global Development Group. (2024). PostgreSQL Documentation. <https://www.postgresql.org/docs/>
- pgvector Contributors. (2024). pgvector: Open-source vector similarity search for Postgres. <https://github.com/pgvector/pgvector>
- Malkov, Y. A., & Yashunin, D. A. (2018). Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs. IEEE Transactions on Pattern Analysis and Machine Intelligence, 42(4), 824-836.
- Lewis, P., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. Advances in Neural Information Processing Systems (NeurIPS).